



(Scan to read the Practice assignment)

Software Architecture Design

Question 1: Business Requirement

I represent an event management company, and we would like to build an online ticketing system to serve both customers and our internal staff. Our customers need a mobile application where they can browse ongoing events, choose seats, purchase tickets, make payments, and review their previous purchases. For staff and managers, we need a desktop-based system that allows them to update event information, arrange seating layouts, track ticket sales, and manage users. All user requests should be received at a single entry point and then automatically routed to the appropriate components for processing. Within the system, we want to separate each functional area to make future management and expansion easier. For example, there should be a dedicated component for handling authentication and access control, one for managing event content and seating, one for ticket booking, one for storing customer information and purchase history, one for payment processing, and one for sending notifications to customers after successful purchases or when an event is about to start. Because the number of users may be very high during peak times, the system must be able to automatically distribute processing workloads to prevent overload. At the same time, we want a protection layer to block invalid external access. Data for each functional component should be stored separately so that upgrading or maintaining one part does not interrupt the others. In addition, we need the ability to monitor the overall health of the system in order to quickly detect issues and evaluate system stability.

Student requirement: Based on the customer description above, analyze the system and design a high-level block diagram that shows user groups, main functional components, processing flows, and data storage.

Question 2: (Clean Code & Refactoring)

Given the following code:

```
public class SystemManager
{
    public void DoEverything(string type, double money)
    {
        if(type == "card")
        {
            Console.WriteLine("Pay with card: " + money);
        }
        else if(type == "paypal")
        {
            Console.WriteLine("Pay with paypal: " + money);
        }

        if(money > 500)
            Console.WriteLine("Send VIP email");
        else
            Console.WriteLine("Send normal email");
    }
}
```

- Tasks:**
1. Identify at least **3 code smells**.
 2. Refactor the code following Clean Code principles:
 - Meaningful naming
 - Separate responsibilities
 - Remove magic strings
 - Reduce coupling
 3. Briefly explain your refactoring decisions.

Question 3: (OOP + SOLID)

The system currently supports:

- Card payment
- Paypal payment

Upcoming payment methods:

- Momo
- ZaloPay

- Tasks:**
1. Redesign the payment module using OOP so that:
 - It follows Open/Closed Principle
 - It follows Dependency Inversion
 - New payment methods can be added without modifying existing classes
 2. Draw a simple UML class diagram or describe your class structure.

Question 4: (Design Patterns)

The notification system must support:

- Email
- SMS
- Push Notification

Tasks

Students must create a **C# (.NET) project** implementing:

Required components:

- INotifier
- EmailNotifier
- SmsNotifier
- PushNotifier
- NotifierFactory

Apply:

- Factory Pattern
- Strategy Pattern
- Decorator Pattern (Logging OR Retry)

Project Requirements

- Students must:
1. Create a **.NET Console Application**
 2. Students must organize the project structure by themselves.

The folder organization should clearly reflect:

- Separation of concerns
- Applied design patterns
- Readability and maintainability

There is **no fixed folder structure provided**. Students are required to design their own project layout.

The structure must demonstrate:

- Clear separation between interfaces, implementations, factory, and decorators
- Logical grouping of related classes
- Meaningful folder and file naming

Students must briefly explain their folder structure and design decisions in **README .md**.

2. Demonstrate usage in **Program .cs**